— EVALUACIÓN PARCIAL N.° 2

MesaTech Cloud

MENSAJERÍA Y PERSISTENCIA INDEPENDIENTE



Ari Araya · Nicolás Barra · Giannina Guerrero · Skarlett Tropan

DOCENTE
Paulo Andrés Berríos Díaz

— QUÉ CAMBIÓ RESPECTO DE LA EP1

De una consulta en línea a un evento.

Solicitudes ya no depende de que Catálogo responda.
Recibe la categoría por mensaje y guarda su propia copia.

Bajo acoplamiento
Catálogo publica y continúa. No espera a Solicitudes.

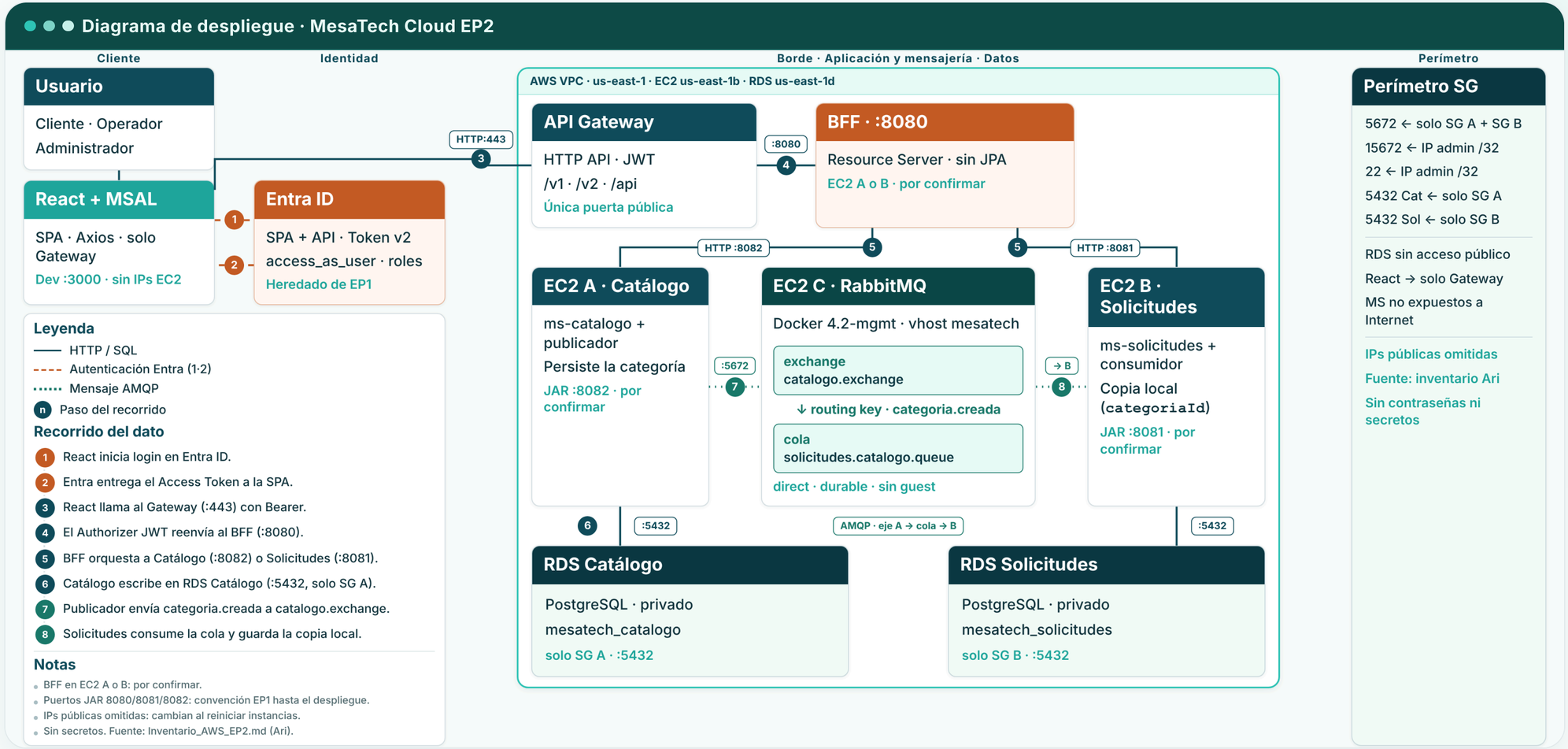
Base de datos por servicio
Cada microservicio escribe solo en su RDS. Ninguno lee la base del otro.

Consistencia eventual
La copia llega con un breve retraso. La categoría se ofrece cuando ya fue consumida.



— DESPLIEGUE EP2 · US-EAST-1

Tres EC2, dos RDS y un broker entre ambos servicios.



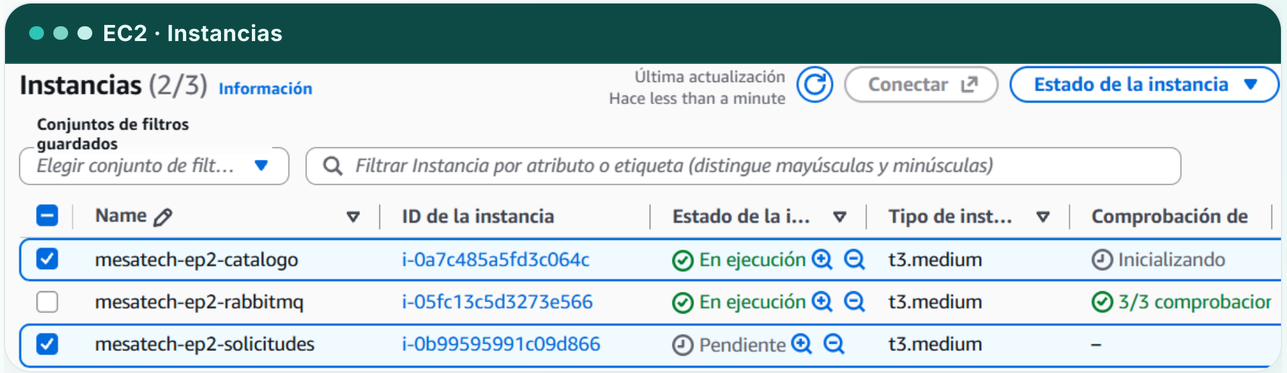
Línea continua: HTTP y SQL. Punteada: mensaje AMQP de A a C y de C a B. En naranjo, Entra ID y BFF, heredados de la EP1.

— AWS ACADEMY · US-EAST-1

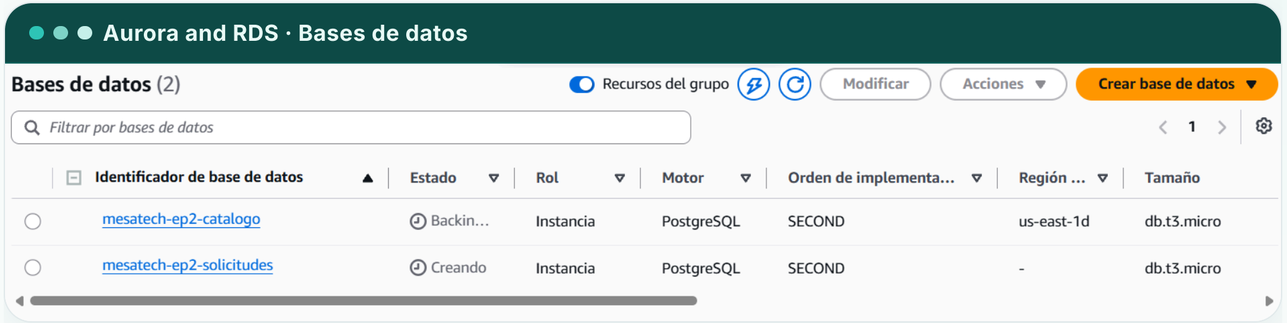
Cada servicio en su máquina y con su base.

Separadas, una falla o un reinicio no arrastra a los demás. Cada máquina tiene su propio security group.

EC2 A	**mesatech-ep2-catalogo**	Catálogo y publicador · OpenJDK 17
EC2 B	**mesatech-ep2-solicitudes**	Solicitudes y consumidor · OpenJDK 17
EC2 C	**mesatech-ep2-rabbitmq**	Broker RabbitMQ · Ubuntu 24.04 LTS
2 RDS	**PostgreSQL · db.t3.micro**	Una por servicio · acceso público: No



Tres instancias t3.medium distintas: Catálogo, RabbitMQ y Solicitudes.



Dos RDS PostgreSQL exclusivas. Captura tomada durante la creación.

— EC2 C · DOCKER COMPOSE

Broker instalado desde cero, con versión fija.

SISTEMA	DOCKER ENGINE	COMPOSE	IMAGEN
Ubuntu 24.04 LTS	29.9.2	v5.6.1	rabbitmq:4.2-management

hostname mesatech-rabbitmq · 5672 · AMQP · 15672 · consola · volumen /var/lib/rabbitmq · restart: unless-stopped · credenciales en .env fuera de git

EC2 C · docker compose pull · up -d · ps

```
docker compose pull
docker compose up -d
docker compose ps
docker compose logs --tail=30
[+] pull 14/14
 ✔ Image rabbitmq:4.2-management Pulled                                                
```

— SECURITY GROUPS

Cada puerto se abre solo a quien lo necesita.

DESTINO	PUERTO	ORIGEN PERMITIDO	PARA QUÉ
EC2 C · RabbitMQ	5672 AMQP	SG de EC2 A y SG de EC2 B	Publicar y consumir
EC2 C · RabbitMQ	15672 consola	IP de administración /32	Administrar el broker
EC2 A, B y C	22 SSH	IP de administración /32	Operar las instancias
RDS Catálogo	5432	Solo SG de EC2 A	Solo Catálogo escribe
RDS Solicitudes	5432	Solo SG de EC2 B	Solo Solicitudes escribe

RDS sin acceso público
Se consultan solo desde su EC2.

Origen por security group
La regla sigue valiendo si cambia la IP de la instancia.

Sin usuario guest
Un usuario por aplicación en el broker.

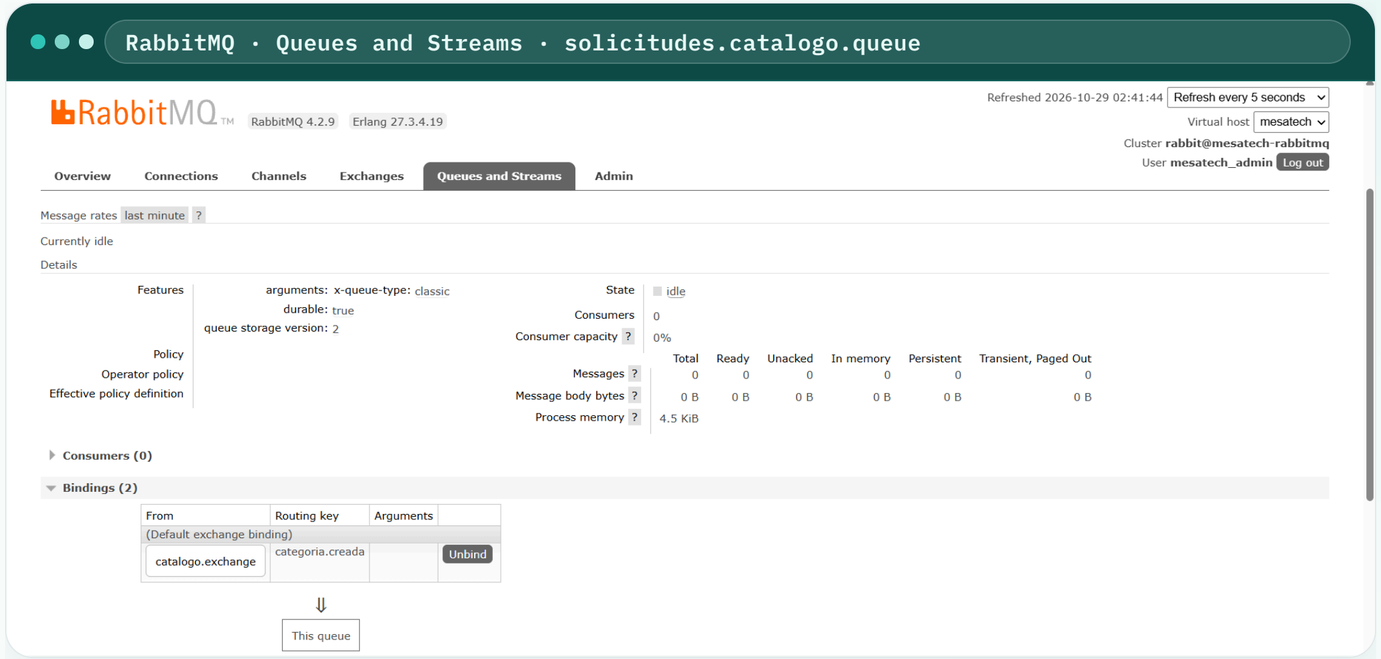


— RECURSOS DEL BROKER

Un exchange, una cola y una routing key.

VHOST	**mesatech** Aísla los recursos del proyecto
EXCHANGE	**catalogo.exchange** direct · durable
ROUTING KEY	**categoria.creada** Enlaza el exchange con la cola
COLA	**solicitudes.catalogo.queue** durable · classic
USUARIOS	**catalogo_publicador** · **solicitudes_consumidor** Uno por aplicación

[Captura pendiente: permisos de ambos usuarios sobre el vhost mesatech]



Cola durable en el vhost mesatech, enlazada a catalogo.exchange con categoria.creada.

— CONTRATO DEL EVENTO · V1.0

Cinco campos que ambos servicios respetan.

CAMPO	TIPO	REGLA
`eventId`	String	Obligatorio, no vacío
`tipoEvento`	String	Siempre `categoria.creada`
`fechaEvento`	Instant	ISO-8601 en UTC
`categoriaId`	Long	Mismo id de RDS Catálogo
`nombre`	String	Obligatorio; se guarda sin espacios sobrantes

JSON · routing key categoria.creada — EJEMPLO

```json
{
  "eventId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "tipoEvento": "categoria.creada",
  "fechaEvento": "2026-10-29T15:00:00Z",
  "categoriaId": 7,
  "nombre": "Hardware"
}
```

> Ejemplo ilustrativo, no captura de un mensaje real. Un evento incompleto o de otro tipo se rechaza sin reencolar.

— PUBLICADOR Y CONSUMIDOR

Dos confirmaciones distintas.

Publicador · Catálogo

Publisher confirm: el broker recibió el mensaje. Returned: no había cola para la routing key.

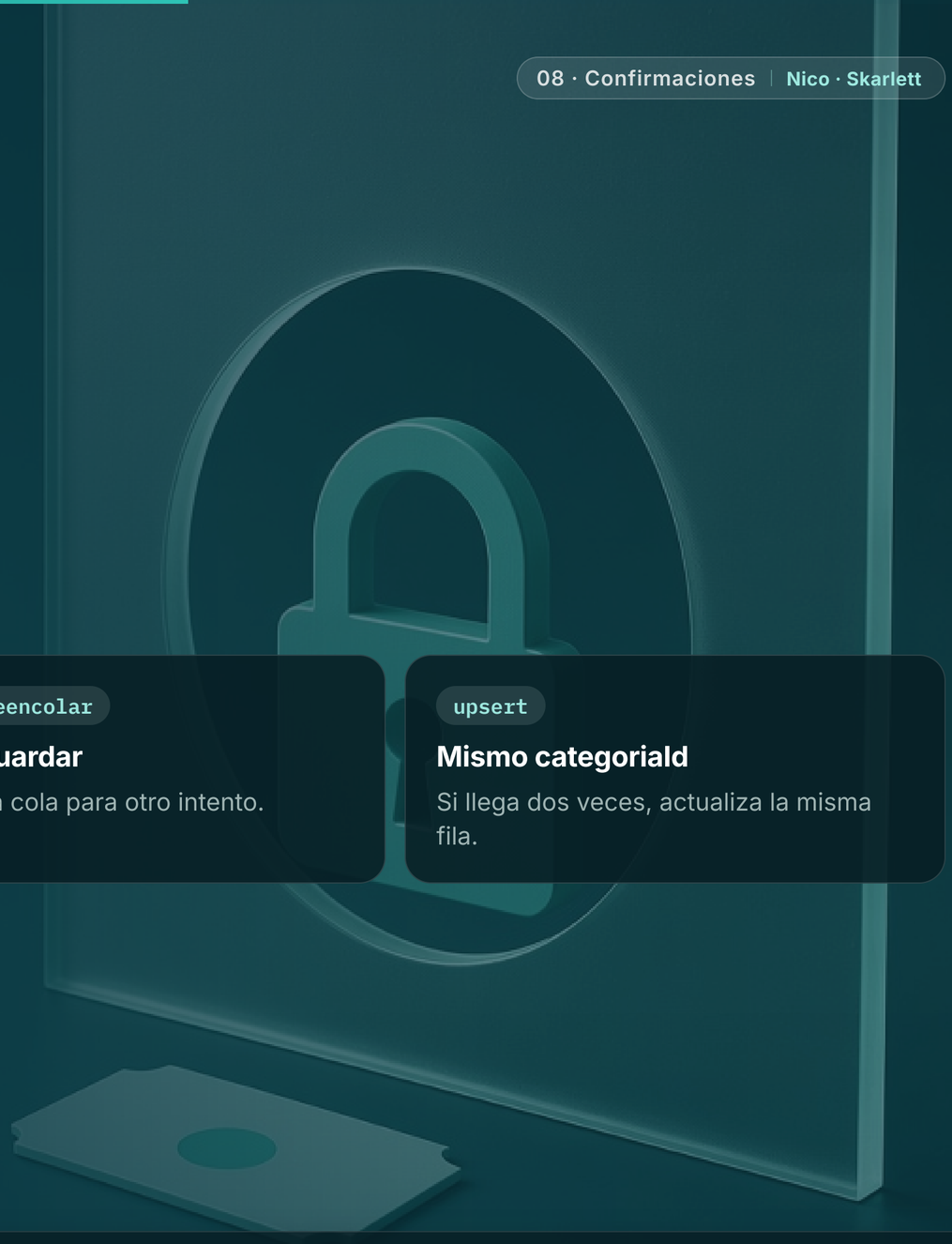
En desarrollo · Nicolás

→

Consumidor · Solicitudes

Ack manual, enviado solo después de guardar la copia en RDS Solicitudes.

En código · Skarlett



`basicAck`

Copia guardada

El mensaje sale de la cola.

`nack · sin reencolar`

JSON inválido o incompleto

Se descarta: reintentar no lo arregla.

`nack · reencolar`

Falla al guardar

Vuelve a la cola para otro intento.

`upsert`

Mismo categoriald

Si llega dos veces, actualiza la misma fila.

Consumidor: rama `feature/skarlett-consumidor` · `CategoriaCreadaListener` y `CategoriaCopiaService`

— CONTINUIDAD DE LA EP1

La puerta de entrada no cambió.

FRONT	**React + MSAL**	Inicio de sesión y Access Token v2
IDENTIDAD	**Microsoft Entra ID**	Roles Cliente, Operador y Administrador
BORDE	**API Gateway**	Única puerta pública · JWT Authorizer
ORQUESTA	**BFF :8080**	Valida token y rol; sin base de datos
CONTRATOS	**v1 y v2**	`/v1/solicitudes/mias` · `/v2/solicitudes/mias`

RUTA NUEVA

`GET /v1/solicitudes/categorias`

Lista las categorías ya recibidas en la copia local. Se expone en API Gateway, igual que el resto de `/v1/solicitudes`.

[Captura pendiente: P8 · inicio de sesión, 401 sin token y 200 vía Gateway sobre la topología EP2]

— PRUEBAS P1 A P8

Ocho pruebas definidas para AWS.

P1 Pendiente de ejecución

Separación real de EC2 y RDS

P2 Pendiente de ejecución

Flujo completo hasta la copia local

P3 Pendiente de ejecución

Consumidor detenido: mensaje en Ready

P4 Pendiente de ejecución

Consumo diferido con ack

P5 Pendiente de ejecución

Reinicio del broker con volumen

P6 Pendiente de ejecución

Reinicio de apps sin pérdida

P7 Pendiente de ejecución

Broker detenido al publicar

P8 Pendiente de ejecución

Continuidad EP1: 401, 403, v1/v2

ENTORNO LOCAL Consumidor probado fuera de AWS: evento con `categoriaId 42`, «Redes-local», guardado en `categorias_locales` con el mismo id. Además, pruebas unitarias de ack, nack y conservación del id.

— ESTADO Y PRÓXIMOS PASOS

Lo que está listo y lo que falta cerrar.

LISTO

Infraestructura AWS
3 EC2, 2 RDS y security groups · Ari

RabbitMQ en Compose
Vhost, exchange, cola y usuarios · Ari

Consumidor en código
Ack manual y copia local · Skarlett

Contrato y diagrama
Evento v1.0 y despliegue EP2 · Giannina

PRÓXIMOS PASOS

1. **Publicador en ms-catalogo**
   Confirms y returned · Nicolás
2. **JAR de Solicitudes en EC2 B**
   Con su RDS y el broker · Skarlett
3. **Ruta en API Gateway**
   `GET /v1/solicitudes/categorias` · Giannina
4. **Ejecución de P1 a P8**
   Con capturas en cada carpeta · Equipo

— RECORRIDO DEL DATO

De React a la copia en RDS Solicitudes.

01 **React → API Gateway**
Petición con Access Token de Entra ID

02 **Gateway → BFF**
JWT Authorizer valida y reenvía a :8080

03 **BFF → Catálogo**
Revisa el rol y llama a ms-catalogo

04 **Catálogo → RDS Catálogo**
Guarda la categoría y obtiene su id

05 **Catálogo → exchange**
Publica categoria.creada en catalogo.exchange

06 **Exchange → cola**
La routing key lo deja en solicitudes.catalogo.queue

07 **Cola → Solicitudes**
El listener recibe el JSON del evento

08 **Solicitudes → RDS Solicitudes**
Guarda la copia con el mismo id y envía ack



— EVALUACIÓN PARCIAL N.° 2

Lo que sostiene la arquitectura.

01 **Separación**
Tres EC2 y dos RDS exclusivas. Ningún servicio lee la base del otro.

02 **Mensajería**
Evento categoria.creada por RabbitMQ, con ack después de guardar.

03 **Continuidad**
Entra ID, API Gateway y BFF siguen siendo la entrada segura.

PREGUNTAS



Ari
AWS y RabbitMQ

Giannina
Diagrama, contrato y continuidad

Nicolás
Publicador y Catálogo

Skarlett
Consumidor y pruebas

DOCENTE
Paulo Andrés Berríos Díaz